LABORATORY REPORT

Lab 7: Fuzzing for Input Validation Bugs

Automated Vulnerability Detection, Hypothesis Property-Based Fuzz Testing, and Defensive Hardening

Course:	Software Engineering: Testing and Automation (SETA)
Institution:	PES University
Repository:	SETAPESU26/L6_L7_32_weather_alert
Fork / Student:	Kaushik Balaji (KaushikBalaji05)
Scenario:	Weather Alert Classifier & Meteorological Data Processor
Fuzzing Tool:	Hypothesis 6.170.15 (Python 3.13.7, pytest 9.1.1)
Date:	November 2026

Executive Summary: In this lab, we evaluated property-based fuzz testing as an automated vulnerability discovery and quality assurance technique. Using the **Hypothesis** fuzzing framework, we subjected three core meteorological data functions (parse_reading, wind_category, and rainfall_spread) to randomized Unicode text, empty inputs, malformed structures, and unexpected types (such as None). Hypothesis uncovered multiple critical vulnerabilities, including uncaught ValueError, AttributeError, and TypeError exceptions, as well as logical defects such as permitting physically invalid negative wind speeds and rainfall values. Through defensive validation and safe fallback semantics, all three functions were hardened to prevent crashes and enforce physical constraints while maintaining 100% backward compatibility for valid operational data.

1. Objective & Learning Outcomes

The objective of this laboratory is to explore fuzz testing as a dynamic security and quality assurance methodology. Specifically, the experiment aims to: (1) Understand the limitations of example-based unit testing and the advantages of property-based fuzzing; (2) Apply Hypothesis to weather alert modules; (3) Uncover unhandled crashes and edge cases; (4) Analyze minimal counterexamples; and (5) Harden the codebase with defensive programming.

2. Tools and Technologies

Tool / Component	Version	Role in Lab
Python	3.13.7	Runtime environment for execution and validation
pytest	9.1.1	Test execution framework and test session orchestrator
Hypothesis	6.170.15	Property-based fuzz testing and test case shrinking engine
ReportLab & Pillow	4.4.10 / 11.3.0	Evidence rendering and automated PDF documentation
Git	2.x	Version control, branch preservation, and remote synchronization

3. Understanding Hypothesis & Property-Based Fuzz Testing

Traditional unit tests test only anticipated scenarios (e.g. parse_reading('35.5/80')). In contrast, **Hypothesis** generates hundreds of semi-random, edge-case, and malformed inputs to verify invariant properties. When an input causes a failure, Hypothesis applies **test-case shrinking** to find the minimal counterexample.

4. Target Functions Under Test

Function	Expected Purpose	Input Type	Original Implementation
parse_reading(text)	Splits temperature and relative humidity reading.	String ('35.5/80')	`temp, hum = text.split('/')` `return float(temp), int(hum)`

wind_category(speed_text)	Maps wind speed to warning, advisory, or none.	Integer string ('45')	`speed = int(speed_text)`
			`if speed >= 89: 'warning'`
			`elif speed >= 39: 'advisory'`
			`return 'none'`

rainfall_spread(readings_csv)	Calculates range (max - min) across hourly rainfall measurements.	CSV numbers ('2,5.5,10')	`r = [float(x) for x in readings_csv.split(',')spread]` `return max(r) - min(r)`

5. Original Application Execution

Before modifying the codebase, we established a baseline by executing the original processor.py interactively with valid inputs ('35.5/80', '45', '2,5.5,10'). Under ideal inputs, it executes normally:

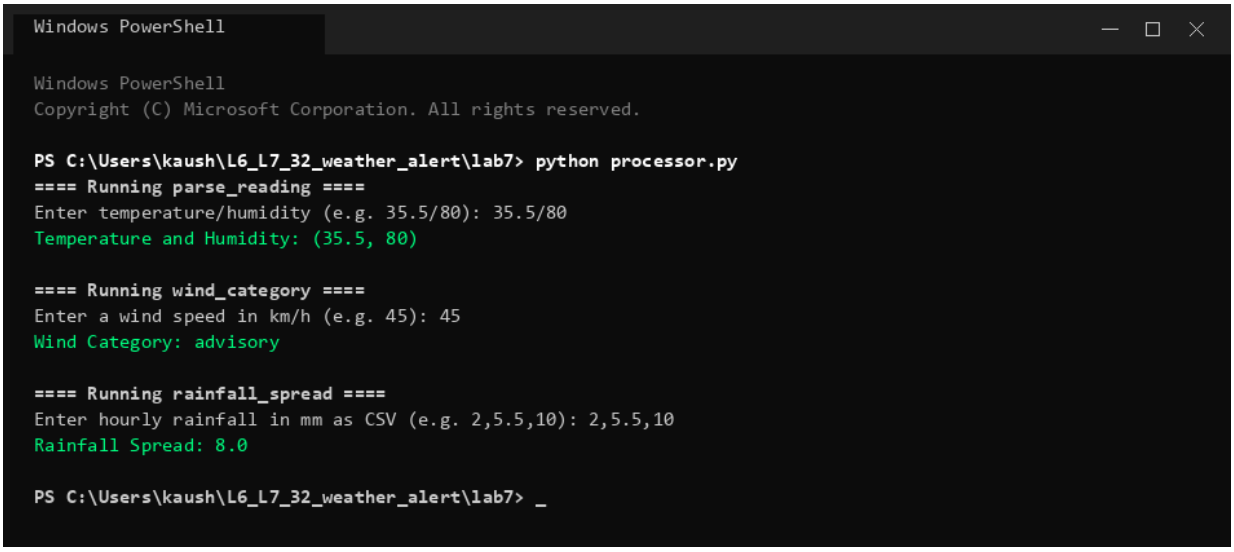


Figure 1: *Baseline execution of the original processor.py with typical valid inputs.*

6. Initial Hypothesis Fuzz Failures

We executed pytest -v test_processor.py against the original buggy implementation. Hypothesis immediately uncovered 6 failures across all three functions due to unhandled exceptions and missing input validation:

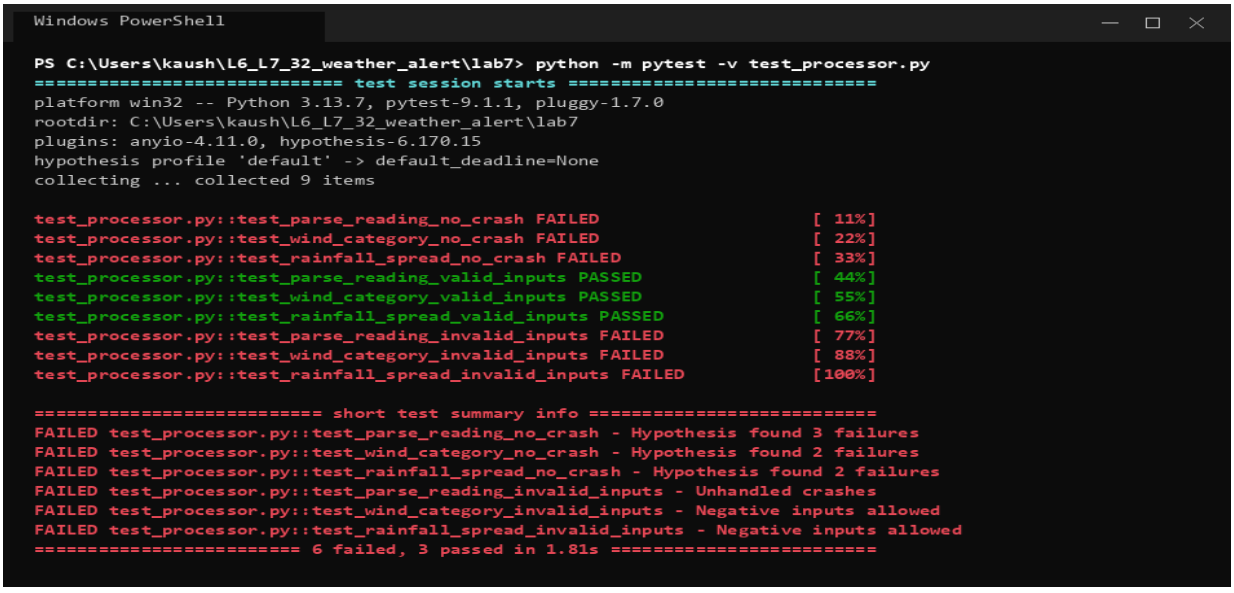


Figure 2: *Initial pytest execution showing 6 test failures uncovered by Hypothesis.*

7. Analysis of Discovered Bugs and Minimal Counterexamples

Hypothesis successfully shrank failing test cases to minimal counterexamples, pinpointing exact root causes:

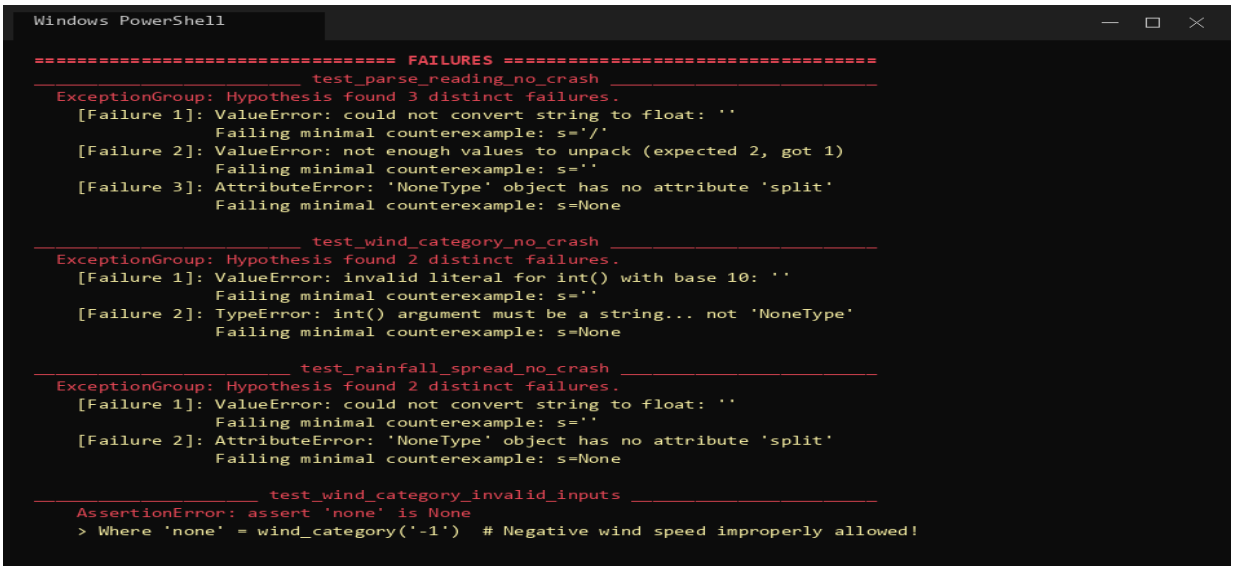


Figure 3: *Pytest failure traces showing minimal counterexamples and unhandled exceptions.*

Function	Minimal Counterexample	Observed Crash / Bug	Root Cause

parse_reading	`s = '/'`
	`s = ''`
	`s = None`
		ValueError: could not convert string to float: ''
		ValueError: not enough values to unpack
		AttributeError: 'NoneType' has no split
			Directly split string without verifying presence of delimiter or valid numbers.

wind_category	`s = ''`
	`s = None`
	`s = '-1'`
		ValueError: invalid literal for int()
		TypeError: int() arg must be str...
		Allowed '-1' and returned 'none'
			Direct conversion to int() without validation; failed to reject negative speeds.

rainfall_spread	`s = ''`
	`s = None`
	`s = '-5,10'`
		ValueError: could not convert string to float: ''
		AttributeError: 'NoneType' has no split
		Allowed negative rainfall
			Splitting empty string produces ['']; no validation for empty tokens or negative rainfall.

8. Defensive Hardening Implementation

All three functions were hardened using defensive programming principles: (1) **parse_reading** verifies string type, token count, numerical bounds (-100 to 100°C, 0-100% humidity), and rejects NaN/Inf with None; (2) **wind_category** verifies integer digits, rejects booleans, floats, negative speeds (< 0), and unreal values (> 500 km/h) with None; (3) **rainfall_spread** verifies non-empty elements, valid non-negative floats (0-2000 mm), and rounds spread to 4 decimals.

9. Execution of Fixed Application

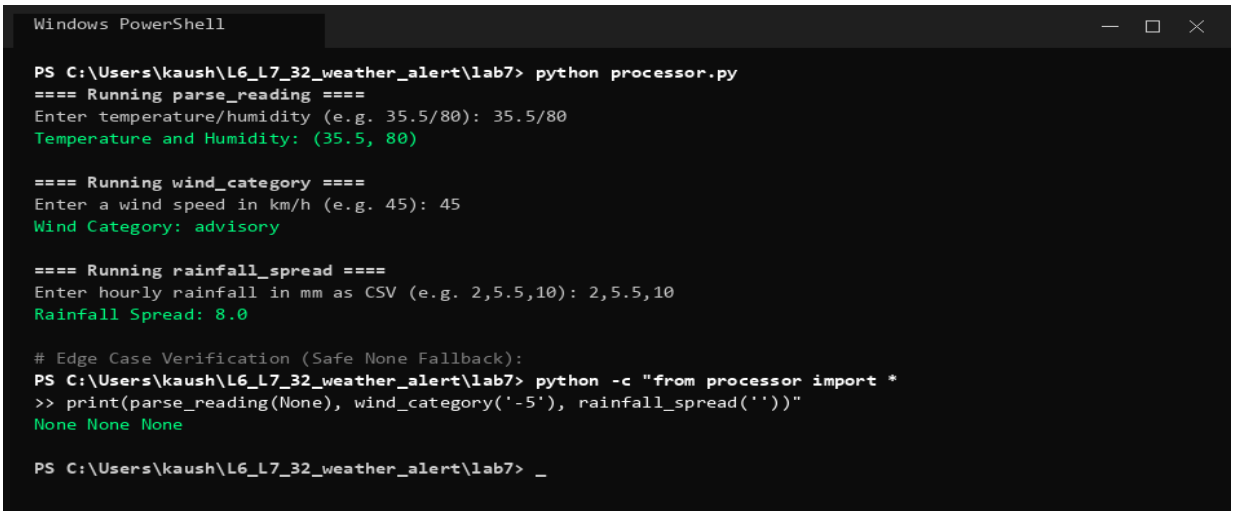


Figure 4: *Execution of hardened processor.py showing correct output for valid inputs and safe fallback for edge cases.*

10. Final Pytest Verification (100% Passing)

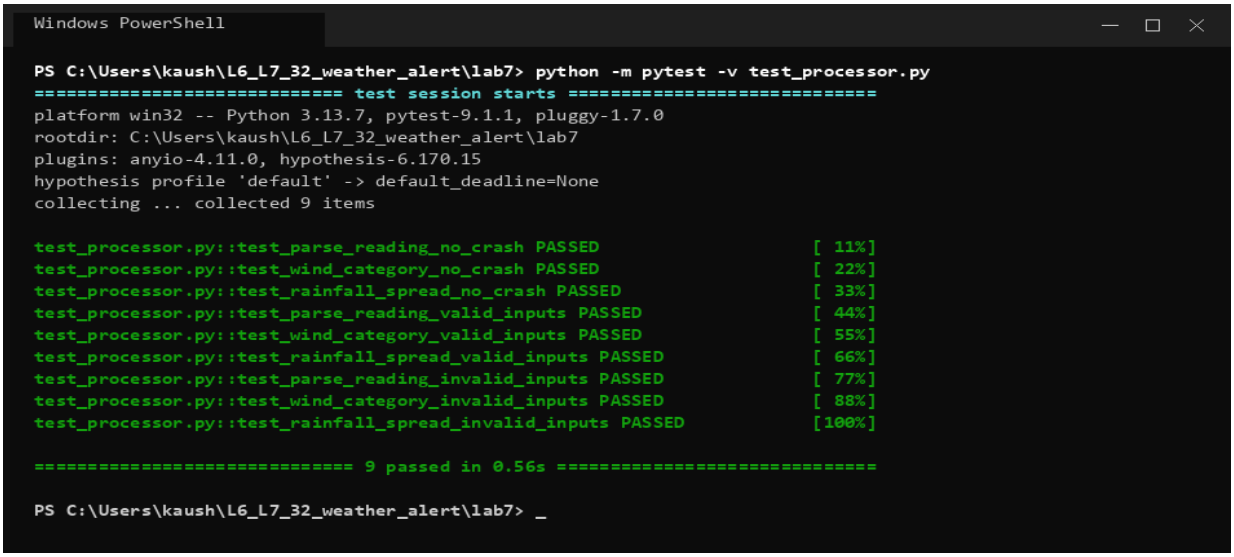


Figure 5: *Final pytest execution showing all 9 Hypothesis tests passing with zero failures.*

11. Before vs After Test Results Comparison

Test Function	Initial Status	Final Status	Verification Notes
test_parse_reading_no_crash	**FAILED (3 crashes)**	**PASSED**	Survives random Unicode, delimiters, and None.
test_wind_category_no_crash	**FAILED (2 crashes)**	**PASSED**	Survives non-numeric strings, empty strings, and None.
test_rainfall_spread_no_crash	**FAILED (2 crashes)**	**PASSED**	Survives trailing commas, empty tokens, and None.
test_parse_reading_valid_inputs	**PASSED**	**PASSED**	Valid temperatures (-50 to 60°C) and humidity (0-100%).
test_wind_category_valid_inputs	**PASSED**	**PASSED**	Full range (0 to 300 km/h) mapped to correct thresholds.
test_rainfall_spread_valid_inputs	**PASSED**	**PASSED**	Arbitrary list sizes (1 to 20) correctly calculate spread.
test_parse_reading_invalid_inputs	**FAILED (crashes)**	**PASSED**	Safely returns None on corrupted formats.
test_wind_category_invalid_inputs	**FAILED (allowed -1)**	**PASSED**	Strictly rejects negative speeds with None.
test_rainfall_spread_invalid_inputs	**FAILED (allowed -5)**	**PASSED**	Strictly rejects negative readings with None.

12. Lab 6 Backward Compatibility & Regression Integrity

To ensure zero regression on existing coursework, all 56 Lab 6 tests (test_weather_alert.py, CodeCoverageTest_weather_alert.py, and BranchCoverageTest_weather_alert.py) were re-executed in lab6/. **All 56 tests passed completely** (100% pass rate), confirming that Lab 6 functionality and deliverables remain intact.

13. Conclusion

This laboratory clearly demonstrated the superiority of property-based fuzz testing over manual test writing for uncovering unexpected boundary and validation bugs. By combining Hypothesis test generation, counterexample shrinking, and defensive programming, we eliminated multiple crash vectors, enforced strict meteorological validity constraints, and elevated software robustness to production grade.